

XXVI SIMPÓSIO BRASILEIRO DE CIBERSEGURANÇA · SBSEG 2026

1 a 4 de setembro de 2026 · Armação dos Búzios, RJ



IoTedu Core: correlação multi-IDS e contenção automatizada de ataques em redes IoT institucionais



Emanuel Ferreira · Matheus Ciocca · Douglas Fideles · Silvio Quincozes · Diego Kreutz

AI Horizon Labs · Universidade Federal do Pampa (UNIPAMPA)

Programa de Pós-Graduação em Engenharia de Software (PPGES)

O problema



Detectar não é conter.

E conter no alerta errado tira do ar um dispositivo legítimo.

Alerta não é ação

O IDS detecta, mas o bloqueio segue manual.
[Waleed et al. 2022]

Cobertura parcial

Cada motor cobre um subconjunto das classes.
[Boukebous et al. 2023]

Custo invisível

Ninguém reporta quantos dispositivos legítimos o bloqueio derruba. [Wazuh 2026]

Nenhum IDS sozinho basta

- Cada IDS é cego em um lugar diferente.
- Concordar entre IDS custa cobertura.
- Errar a família não é o mesmo que não detectar.

Medição própria: 240 janelas rotuladas, 15 por cenário.

	Snort	Suricata	Zeek
ICMP Flood	1,00	1,00	0,00
SYN Flood	1,00	1,00	0,33
UDP Flood	1,00	1,00	0,47
DoS HTTP	0,33	0,73	0,00
TCP Scan	0,00	0,00	0,73
SSH Brute F.	0,00	0,00	0,00
DNS Tunnel	0,27	1,00	0,67
SQLi	0,00	1,00	0,73
XSS	1,00	1,00	0,40
Path Trav.	0,00	0,00	0,67

O que já existe e o que falta

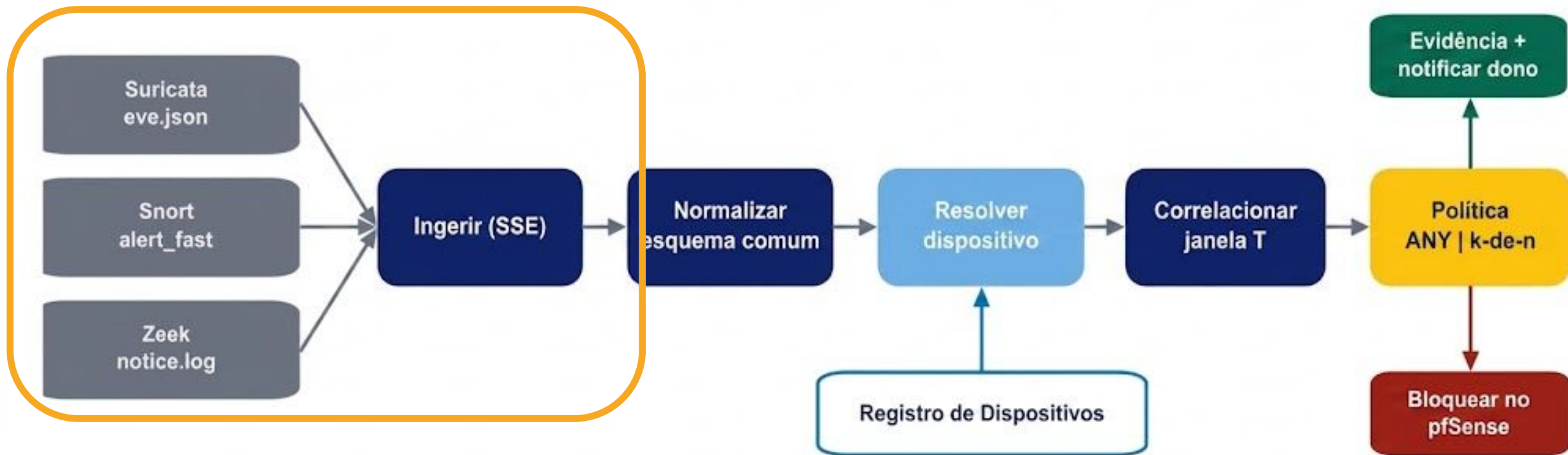
Abordagem	Operação	O que falta medir
Avaliações comparativas	Só detecção	Não integram alertas nem contêm
Plataformas de integração	Só detecção	Agregam eventos, mas não bloqueiam
Abordagens que bloqueiam	Detecção + resposta	Um único motor, sem correlação
SIEM / XDR (Wazuh, Security Onion)	Detecção + resposta	Não reportam bloqueio indevido
IoTedu Core (este trabalho)	Detecção + resposta	Mede cobertura e bloqueio indevido

O pipeline já existe. A evidência sobre o que ele custa, não.

01 A ferramenta

Do alerta de três motores ao bloqueio por dispositivo

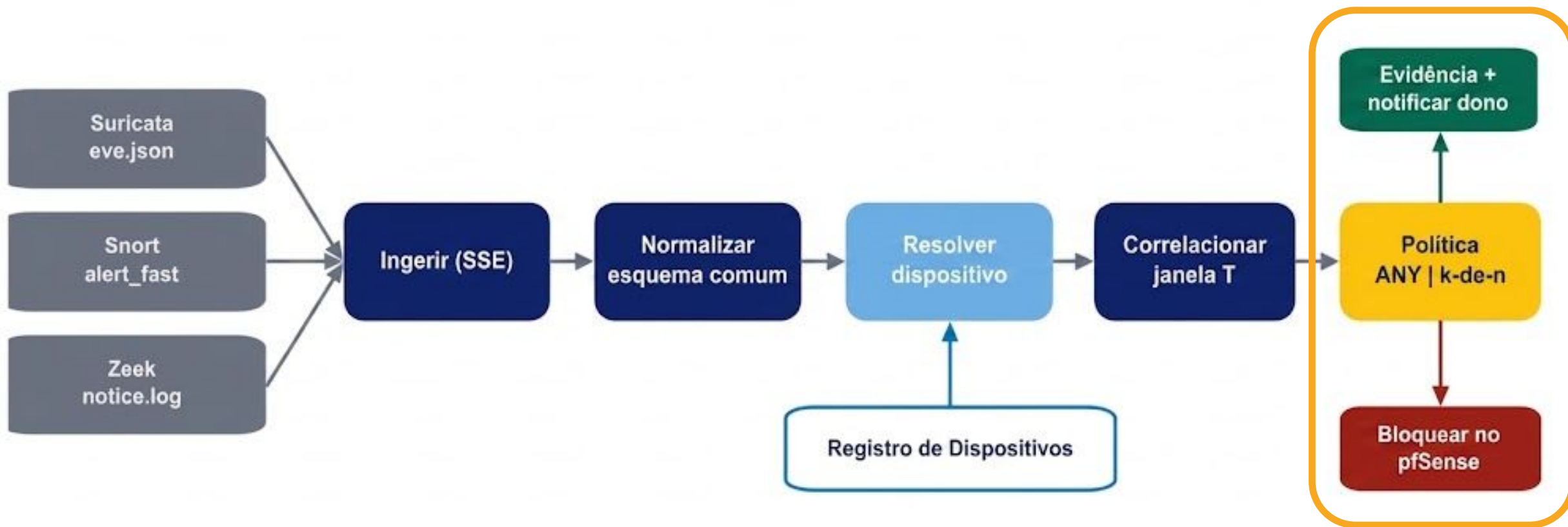
Do alerta ao bloqueio: coleta



Do alerta ao bloqueio: identidade



Do alerta ao bloqueio: decisão



As duas políticas de contenção

- **ANY: bloqueia no primeiro alerta**
- Basta um motor acusar severidade alta.
- **k-de-n: exige acordo entre IDS**
- Com $k=2$, dois motores apontam a mesma classe no mesmo dispositivo.
- **A política é trocável, sem mexer no pipeline.**

Por que isso importa

ANY é rápido, mas confia em um motor só.

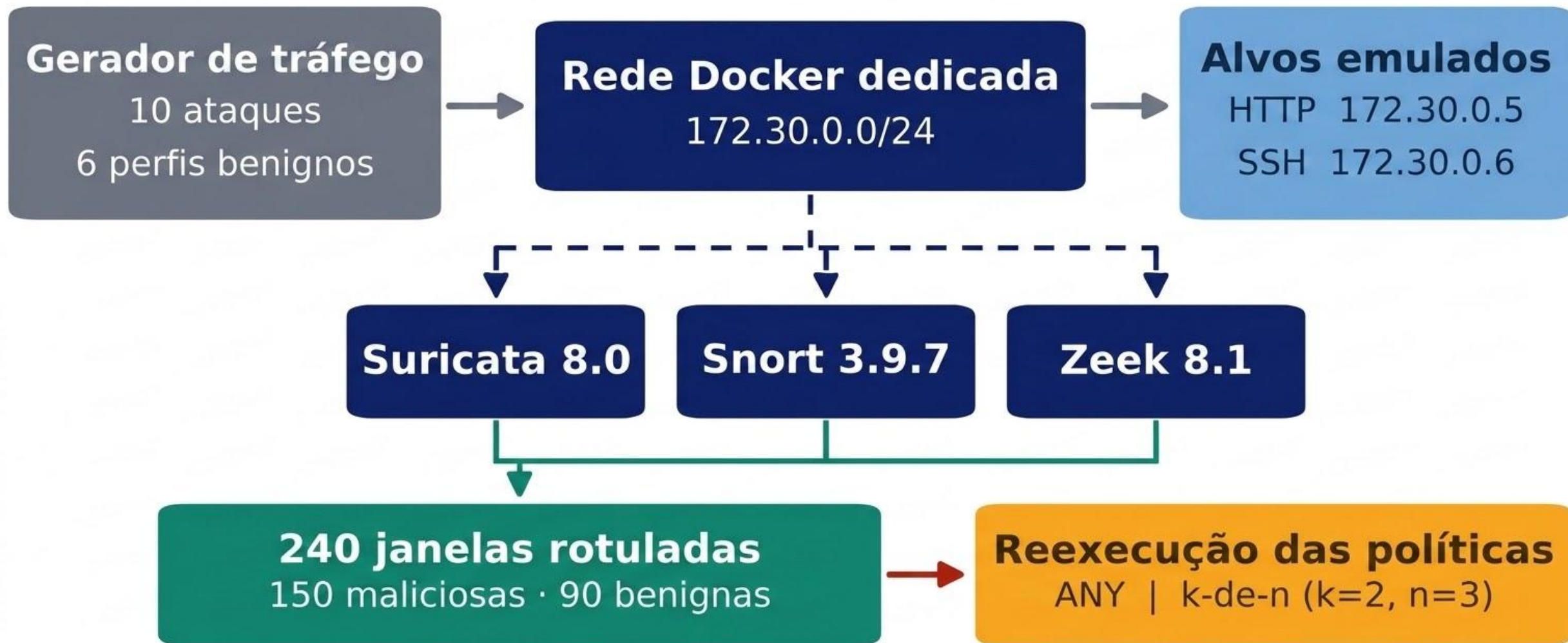
k-de-n demora, mas um falso positivo isolado não derruba ninguém.

Concordar qual ataque é o que o bloqueio exige.

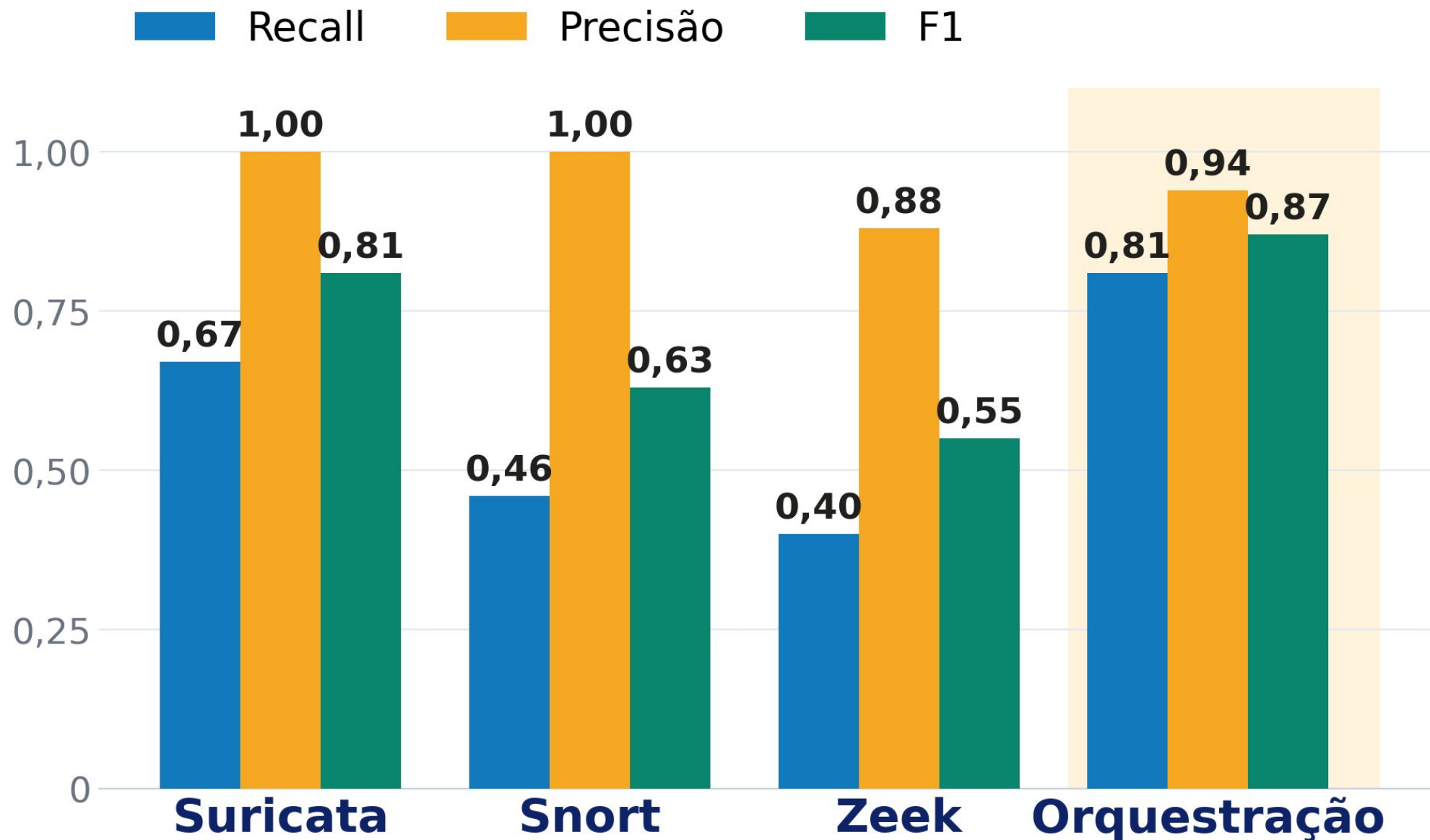
02 Avaliação

O que a orquestração detecta e o que ela derrubaria por engano

A campanha de detecção



Recall 0,81 contra 0,67 do melhor IDS isolado

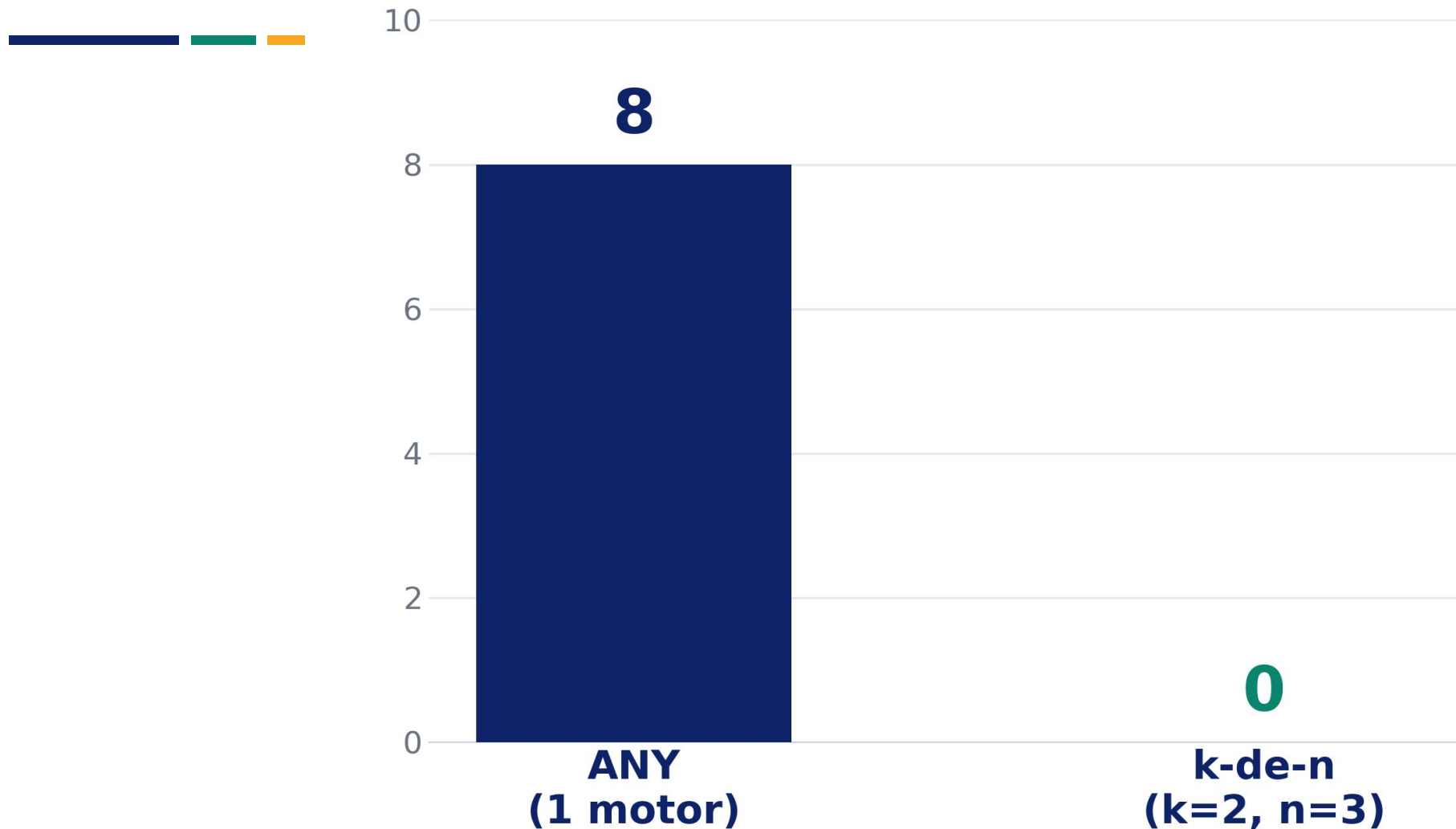


ANY cobre mais classes de ataque



Classes de ataque contidas, das 10 avaliadas na campanha de detecção.

k-de-n elimina os falsos positivos



Bloqueios indevidos, das 90 janelas testadas com tráfego benigno.

A avaliação em três números

0,81

**Recall da
orquestração**

Contra 0,67 do melhor
motor isolado.

0

**Bloqueios
indevidos, $k=2$**

Contra 8 sob a
política ANY.

240

Janelas públicas

Com regras, logs e
scripts abertos.

O que os resultados sustentam

- Mais classes cobertas que qualquer motor isolado.
- Zero bloqueio indevido com $k=2$, independente da janela T.
- O ganho vem da complementaridade entre assinatura e comportamento.

O que ainda não podemos afirmar

- Ambiente controlado, um cenário por janela.
- Latência não instrumentada.
- Contenção por IP de origem, que pode ser forjado.

03

Conclusões

O que fica e para onde vamos

Tornar a orquestração mensurável

Contribuição

Catálogo de regras público e política de contenção trocável.

Evidência

Recall de 0,67 para 0,81, com o custo explicitado: 8 bloqueios sob ANY, zero sob $k=2$.

Impacto

A equipe escolhe entre cobertura e bloqueio indevido com um número na mão.

Trabalhos Futuros

- Taxonomia compartilhada entre os motores.
- Validação em um segmento IoT de campus.
- Avaliar latência e exposição.

Selos Adquiridos



Obrigado!

Perguntas são bem-vindas

AUTOR DE CONTATO

Emanuel Carricio Ferreira

emanuelferreira.aluno@unipampa.edu.br

AI Horizon Labs · UNIPAMPA · PPGES



04

Perguntas Frequentes

Possíveis perguntas da banca e respostas preparadas

Por que não usar diretamente o Wazuh ou o Security Onion, que já integram Snort, Suricata e Zeek?

RESPOSTA:

O Wazuh e o Security Onion são plataformas maduras e excelentes. A questão não é substituí-los.

O que eles NÃO fazem — e que é a nossa contribuição — é reportar o custo operacional da contenção automática. Ou seja: quantos dispositivos legítimos seriam bloqueados sob o mesmo tráfego?

No artigo (Tabela 1), mostramos que nenhum desses trabalhos reporta a coluna FP (false positives sob tráfego benigno) combinada com contenção ativa. A nossa contribuição é justamente tornar essa evidência disponível e a política trocável sem mexer no pipeline.

→ **Em resumo: o pipeline já existe. A evidência sobre o que ele custa, não.**

Por que malicioso e benigno rodam em janelas separadas? Isso não é irrealista?

RESPOSTA:

A separação é uma decisão metodológica deliberada, não uma limitação.

Ao rodar em janelas separadas, conseguimos ATRIBUIR cada falso alarme diretamente ao perfil benigno que o produziu. Se misturássemos, não saberíamos se o alarme veio do tráfego benigno ou do malicioso.

Isso é especialmente importante para o k-de-n: precisamos saber com certeza que o zero de bloqueios indevidos vem de tráfego realmente benigno.

→ **É uma simplificação controlada. A validação em campus (trabalho futuro) vai testar com tráfego misto.**

E a latência? Vocês não mediram o tempo entre o ataque e o bloqueio?

RESPOSTA:

Correto, a latência não foi instrumentada nesta versão. Isso é uma limitação declarada no slide 16 e no artigo.

O foco desta avaliação foi a EFICÁCIA (o que detecta e o que bloqueia por engano), não a eficiência temporal.

A latência depende de vários fatores: janela T, tempo de processamento de cada motor, latência da API do pfSense, e tempo de propagação da regra.

→ **Instrumentar latência e tempo de exposição é o terceiro item dos trabalhos futuros.**

A contenção é por IP de origem. E se o atacante forjar (spoof) o IP?

RESPOSTA:

Essa é uma limitação conhecida e declarada. Se o atacante forja o IP de um dispositivo legítimo, o sistema pode bloquear a vítima.

O k-de-n MITIGA (não elimina) esse risco: para forjar um bloqueio, o atacante precisaria que DOIS motores independentes classificassem o tráfego na mesma classe de ataque.

Nos trabalhos futuros, listamos "identidade mais forte que o IP" — por exemplo, certificado de dispositivo ou autenticação 802.1X — como evolução necessária.

→ **O k-de-n dificulta o ataque, mas não é defesa completa contra spoofing.**

O experimento foi em Docker. Isso escala para uma rede de campus real?

RESPOSTA:

O testbed Docker é um ambiente controlado para a campanha de detecção. O pipeline em si foi projetado para deployment real — inclusive a demonstração (Seção 5 do artigo) roda em VMs com pfSense real.

A escalabilidade depende de três fatores:

- Volume de alertas por segundo (determinado pelos motores, não pelo Core)
- Capacidade da API de normalização
- Capacidade do pfSense de processar regras

→ **A validação em campus é o segundo item dos trabalhos futuros. O objetivo é testar com tráfego real e volume de produção.**

E o SSH brute force? Por que nenhum motor detecta?

RESPOSTA:

O SSH brute force é a única das 10 classes que nenhum dos três motores detectou. Isso é uma limitação do CATÁLOGO DE REGRAS, não da orquestração.

As regras de SSH brute force dos motores dependem de thresholds de tentativas de login em janelas de tempo. No nosso catálogo, essas regras não dispararam para o padrão de tráfego gerado.

Isso mostra que a orquestração só pode recuperar o que pelo menos um motor enxerga. Se nenhum vê, o pipeline inteiro é cego.

→ **A taxonomia compartilhada entre motores (primeiro item dos trabalhos futuros) visa fechar essa lacuna.**

Por que $k=2$ e não $k=3$ (unanimidade)? E se vocês testassem com $k=3$?

RESPOSTA:

Com $k=3$ (unanimidade), exigiríamos que os TRÊS motores concordassem na mesma classe. Na prática, isso eliminaria a maioria das detecções, porque muitas classes são vistas por apenas 1 ou 2 motores.

No mapa de calor (slide 3): varredura TCP e path traversal só o Zeek enxerga. Com $k=3$, essas classes seriam perdidas.

$k=2$ é o menor valor que impede um falso positivo de um único motor de virar bloqueio, sem exigir unanimidade.

→ **$k=2$ com $n=3$ é o ponto de equilíbrio: elimina falsos positivos de motor isolado, preservando a maioria da cobertura.**

Qual o valor de T usado? E se mudar T, os resultados mudam?

RESPOSTA:

Na campanha de detecção, T não é exercitado diretamente. Cada janela roda um cenário isolado com contenção desabilitada.

O replay da política (Seção 4.4 do artigo) agrega por classe de ataque, não dentro de uma janela de T segundos — por isso o k-de-n da avaliação é reportado como LIMITE SUPERIOR.

Na prática, T deve ser pelo menos tão grande quanto a maior janela de agregação interna das regras dos motores. Se T for muito curto, motores lentos não têm tempo de concordar. Se for muito longo, alertas antigos podem influenciar a decisão.

→ **A calibração de T em ambiente real é parte da validação de campus nos trabalhos futuros.**

Os dados e scripts estão disponíveis? Posso reproduzir?

RESPOSTA:

Sim. Tudo está público no repositório GitHub:

- 240 janelas rotuladas com logs originais de cada motor
- Catálogo completo de regras (Suricata, Snort, Zeek) com SHA-256
- Scripts de geração de tráfego e análise
- Vídeo de demonstração

O repositório é: github.com/GT-IoTEdu/iotedu-core_sbseg26

O vídeo está no QR do último slide.

→ **Qualquer pessoa pode baixar, rodar os scripts e reproduzir as tabelas e figuras do artigo.**

Vocês compararam com abordagens baseadas em machine learning? Por que só regras?

RESPOSTA:

O foco do trabalho é a orquestração e a política de contenção, não o mecanismo de detecção em si. Usamos regras porque são determinísticas, auditáveis e reproduzíveis.

Com ML, a detecção melhora potencialmente, mas o problema de CUSTO permanece o mesmo: quantos bloqueios indevidos a política gera?

O IoTedu Core é agnóstico ao mecanismo: se amanhã substituirmos o Zeek por um detector ML, o pipeline de correlação e a política k-de-n funcionam exatamente da mesma forma.

→ **A contribuição é a política mensurável, não o detector.**

O que acontece quando os motores discordam na classe? (ex: Suricata diz SYN flood, Zeek diz scan)

RESPOSTA:

Isso é exatamente o que chamamos de "cross-family misclassification" na Seção 4.3 do artigo.

No k-de-n, a concordância é sobre a CLASSE, não sobre a existência do alerta. Se Suricata diz SYN flood e Zeek diz scan, o k=2 NÃO é satisfeito para nenhuma das duas classes.

Isso é intencional: "concordar que houve ataque é barato; concordar QUAL ataque é o que uma decisão de bloqueio exige."

Na avaliação, identificamos 57 misclassificações cruzadas (Figura 4 do artigo). Exemplo: 15 janelas de path traversal foram classificadas como HTTP DoS pelo Snort e pelo Suricata.

→ **A exigência de concordância de classe torna o k-de-n mais conservador, mas mais seguro.**

Na prática, quem configuraria o k e o T ? O administrador de rede?

RESPOSTA:

Sim, a ideia é que a equipe de rede/segurança escolha a política com base no perfil de risco.

- Rede hospitalar com dispositivos críticos $\rightarrow k=2$ (prioriza evitar bloqueios indevidos)
- Rede de laboratório aberto $\rightarrow ANY$ (prioriza cobertura máxima)

O IoTedu Core torna essa escolha INFORMADA: a equipe sabe, com números, o que cada política custa em cobertura e em bloqueios indevidos.

T é calibrado pelo tempo de resposta dos motores. Na prática, 30–60 segundos seria um ponto de partida razoável.

$\rightarrow$ A contribuição é dar dados para a decisão, não decidir pelo administrador.

Por que só o Zeek gera falsos positivos? Isso não é um problema do Zeek?

RESPOSTA:

O Zeek usa uma abordagem fundamentalmente diferente. Enquanto Snort e Suricata fazem matching de assinaturas (pattern-based), o Zeek usa scripts event-driven que mantêm estado e derivam decisões de observações de rede.

Isso o torna mais sensível — ele detecta classes que os outros não veem (port scan, path traversal), mas também gera alertas em tráfego benigno que se parece com padrão anômalo.

Na avaliação, 8 dos 90 janelas benignas produziram alertas no Zeek (perfis dns, http, icmp, mix, scan). Snort e Suricata: zero.

→ **É justamente essa heterogeneidade que torna o k-de-n efetivo: como os FPs vêm de um só motor, um segundo motor elimina todos.**

Dá para adicionar um quarto motor de detecção? Como?

RESPOSTA:

Sim. A arquitetura é extensível por design.

Para adicionar um novo motor:

1. Escrever um coletor que leia o log do novo motor
2. Mapear os alertas para o esquema comum (Listing 1 do artigo)
3. Empurrar para a API via SSE

A política k-de-n se ajusta automaticamente: com $n=4$, $k=2$ ainda funciona, e pode-se usar $k=3$ para maior rigor.

O esquema comum (source, attack_class, severity) é genérico o suficiente para acomodar motores de ML, WAFs, ou qualquer fonte de alerta.

→ **A normalização para esquema comum é o que permite a extensibilidade.**